

海岛求生

C++ 控制语句应用 — 控制台交互游戏实验

姓 名：张轩基

学 号：1030425213

班 级：计科2502

提交日期：2026.6.26

源代码：<https://github.com/JieMoJi/Island-survival>

while do-while for switch if-else break continue

1. 教材游戏分析 — Pong 游戏

1.1 游戏简介

教材游戏为《现代C++编程实战》第4章实战项目之二——Pong游戏。这是一个经典的双人对战弹球游戏：画面中间有一个球(O)自动运动，左右两侧各有一块挡板(Z)分别由两位玩家通过键盘控制。球碰到挡板反弹并得分，球出界则对方得分，按 Q 键退出游戏。

该游戏使用 C++ 标准库和少量平台 API 实现，核心逻辑约 120 行（不含平台适配代码），展示了控制语句在实时游戏循环中的应用。

1.2 控制语句分析

控制语句	行号	游戏功能	分析说明
while	100	游戏主循环	while(true) 构成无限循环，驱动每一帧的「事件处理→数据更新→画面渲染」三步流程。游戏持续运行直到 break 退出。
if-else if-else	107-123	键盘输入分发	根据按键值分支：W/S 移动左挡板、方向键(72/80)移动右挡板、Q 触发退出。每个分支包含边界检测（挡板不可移出画面）。
if / if-else	130-153	碰撞检测与计分	独立 if 判断球碰上下墙（速度反转）；if-else 判断球碰左/右挡板（反弹+计分）；if 判断球出界（对方得分+球重置）。
for (嵌套)	170-205	帧缓冲绘制	外层 for 遍历行，内层 for 遍历列。每格根据坐标判断绘制什么：O=球，Z=挡板， =中线/边线，-=顶部墙，一次构建整屏画面。
break	122	退出游戏主循环	玩家按 Q 键时，break 跳出 while(true) 主循环，结束程序。是游戏唯一的正常退出路径。
#ifdef / #else	多处	跨平台条件编译	预处理指令区分 Windows（使用 conio.h）和 Linux（手动实现 _kbhit/_getch），确保不同平台的终端兼容性。

1.3 游戏流程图

Pong 游戏简化流程图：



1.4 控制语句汇总

教材游戏使用的控制语句汇总：while (主循环)、if-else if-else (按键分发)、if / if-else (碰撞与计分)、for 嵌套 (帧缓冲渲染)、break (退出循环)、#ifdef/#else (跨平台条件编译)。
未使用：switch、do-while、continue。

2. 自主设计游戏 — 海岛求生 (Island Survival)

2.1 游戏简介

海岛求生是一款自创的回合制生存策略游戏。玩家流落荒岛，需要管理生命值、饱食度、饮水量、士气值四项生存数值。白天从 5 种行动中选择一项执行，夜晚经历随机事件（暴风雨、野兽袭击、星空之夜等），在资源持续衰减的压力下，集齐 3 个无线电零件并搭建 2 级以上庇护所即可点燃信号火获救。

2.2 核心规则

核心规则：

- (1) 四项数值：生命值(100)、饱食度(100)、饮水量(100)、士气值(50)。任一归零则游戏失败。
- (2) 白天行动（五选一）：寻找食物、收集淡水、修建庇护所、探索岛屿、休息恢复。
- (3) 每日衰减：饱食度 -10，饮水量 -15，士气值 -3（迫使玩家策略性选择行动）。
- (4) 夜间事件（4种随机）：暴风雨、野兽袭击、星空之夜、平安夜。庇护所等级影响减伤。
- (5) 胜利条件：集齐 3 个无线电零件（通过探索岛屿随机获得）+ 庇护所 ≥ 2 级。
- (6) 得分公式：存活天数 $\times 100$ + 四项数值 + 获救奖励 500。

2.3 控制语句使用分析

控制语句	出现位置	实现功能
while (3处)	外层重玩循环 / 内层日循环 / 坏输入消费	外层 while(playAgain==1) 控制整局重玩；内层 while(gameOver==0) 驱动每日循环；嵌套 while 配合 cin.get() 消费非法输入字符。
do-while (2处)	行动菜单验证 / 重玩询问验证	先显示菜单再检查输入，保证菜单至少展示一次。cin.fail() + 范围校验确保输入为 1-5 或 0-1。
switch (1处)	白天行动选择 (5 case + default)	根据 1-5 分别执行：寻找食物、收集淡水、修建庇护所、探索岛屿、休息恢复。每个 case 内部用 if-else 实现随机产出。
if-else (10+处)	夜间事件 / 死亡判定 / 胜利判定 / 数值钳	4 分支 if-else 链随机选择夜间事件；多层 if-else 检测四项数值归零并给出不同死因提示；if 检测胜利条件；每次修改数值后 clamp 到 [0,100]。
for (6处)	状态栏文本进度条 / 倒计时动画 / 延时循	4 个 for 循环绘制 [####----] 风格进度条；1 个 for 循环实现 3-2-1 救援倒计时；1 个 for 空循环制造延时效果。
break (8处)	switch case 退出 / 胜利跳出日循环	每个 switch case 末尾 break 防止穿透；胜利后 break 跳出内层 while 循环进入结算。
continue (6处)	死亡后跳过夜间阶段 / 夜间死后跳过胜利	白天行动后若死亡，continue 跳过夜间阶段和胜利检测，直接结算；夜间事件后再次死亡检测同理。

2.4 游戏流程

海岛求生游戏流程：

- 标题画面 → 外层 while(playAgain==1) 重玩循环 → 初始化数据
→ 内层 while(gameOver==0) 日循环：
(1) 显示状态栏（4 个 for 进度条）

(2) do-while 输入验证 → switch 执行白天行动
(3) 每日被动衰减 (hunger-10, thirst-15, morale-3)
(4) if-else 死亡检测 → continue (跳过夜间)
(5) if-else 夜间随机事件 + 数值钳制
(6) if-else 二次死亡检测 → continue
(7) if 胜利检测 → for 倒计时 3-2-1 → break
(8) day++
→ 结算界面 (得分计算+显示)
→ do-while 重玩询问 → 回到外层 while / 退出
→ 结束语 → return 0

3. 两款游戏的控制语句对比分析

教材 Pong 游戏 vs 自主设计海岛求生的控制语句对比：

1. 都使用了 while 作为游戏主循环，但结构不同：
教材用单层 while(true)+break；海岛求生用双层 while（外层重玩+内层日循环）+break+continue，结构更丰富。
2. 教材使用 if-else 处理键盘分支和碰撞判断；海岛求生使用 if-else 处理更大的分支——死亡判定（4种死因）、夜间事件（4种事件，且与庇护所等级交互）、胜利条件。分支的嵌套层级和组合复杂度更高。
3. switch 和 do-while 仅在自主设计游戏中使用：
教材的按键处理用 if-else if（符合实时检测场景），海岛求生的菜单选择用 switch（更清晰的枚举型分支）和 do-while（保证菜单至少显示一次后验证输入）。
4. continue 仅在自主设计游戏中使用：
死亡后 continue 跳过夜间或胜利检测，避免了深层嵌套的 if 块，保持代码扁平化。
5. for 在两款游戏中的用途截然不同：
教材用嵌套 for 构建像素级帧缓冲；海岛求生用 for 绘制文本进度条和倒计时动画。
前者面向像素渲染，后者面向文本显示。
6.
教材未使用的三种控制语句（switch、do-while、continue），自主设计游戏均自然引入，覆盖了章节所有核心语法点。

控制语句	教材 Pong	海岛求生	差异说明
while	(主循环)	(双层: 重玩+日循环)	自主设计结构更复杂
if-else	(按键+碰撞)	(死亡+事件+胜利)	自主设计分支维度更多
for	(嵌套像素渲染)	(进度条+倒计时)	完全不同的应用场景
break	(退出循环)	(case+胜利退出)	自主设计使用场景更多
switch	未使用	(行动菜单)	教材用 if-else 替代
do-while	未使用	(输入验证×2)	教材无需输入验证
continue	未使用	(跳过夜间阶段)	教材无需此模式

4. 游戏运行截图

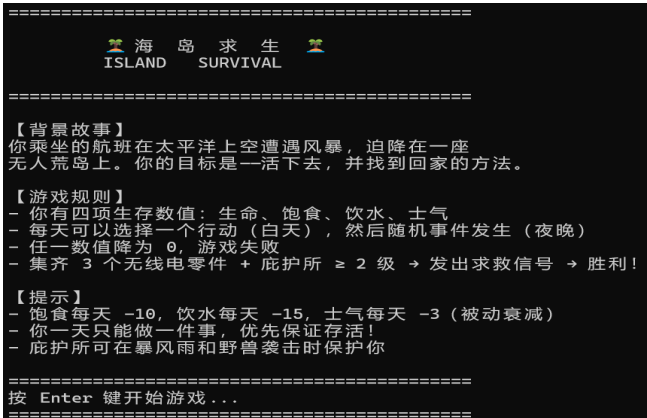


图1: 游戏标题画面与规则说明

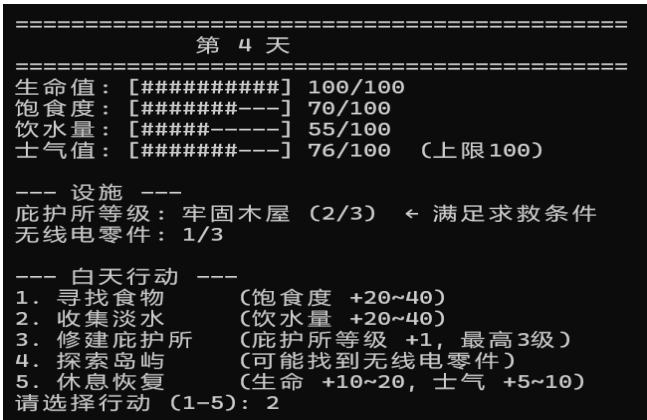


图2: 游戏中 — 状态栏与行动菜单

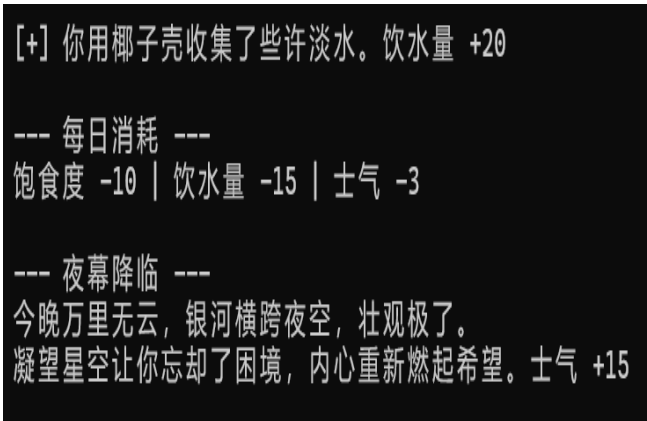


图3: 夜间随机事件

```
=====
你集齐了所有无线电零件！
你的庇护所足够牢固，可以安全组装设备！
你用干燥的木头点燃了求救信号火堆...
=====
等待救援中...
3 ...
2 ...
1 ...
=====
🚁 远处传来了直升机的声音！！！
救援队发现了你的信号火堆！
你得救了！
=====
游 戏 结 束
=====
结 果： 胜 利 ！
=====
存活天数：      5
最终生命值：    82/100
最终饱食度：    50/100
最终饮水量：    25/100
最终士气值：    96/100
庇护所等级：    2/3
无线电零件：    3/3
=====
基础分（天数×100）： 5 × 100 = 500
状态分：          + 243
获救奖励：        + 500
=====
总 分：          1243
=====
再来一局？（1 = 再来一次，0 = 退出）： |
```

图4: 胜利结局 — 获救

```
=====
嘴唇干裂，意识模糊，你因脱水倒下了...
在这片被海水环绕的岛上渴死，多么讽刺。
=====
游 戏 结 束
=====
结 果： 失 败 ...
=====
存活天数：      26
最终生命值：    86/100
最终饱食度：    34/100
最终饮水量：    0/100
最终士气值：    97/100
庇护所等级：    2/3
无线电零件：    2/3
=====
基础分（天数×100）： 26 × 100 = 2600
状态分：          + 217
=====
总 分：          2817
=====
再来一局？（1 = 再来一次，0 = 退出）： |
```

图5: 失败结局 — 数值归零

5. 实验总结与心得

通过本次实验，我对 C++

控制语句的理解从"知道语法"提升到了"知道何时用、如何组合用"的层次。具体收获如下：

(1) 控制语句的选择与场景匹配：

switch 适合固定选项的菜单（如白天行动 1-5），每个 case 语义独立清晰；

if-else 适合条件范围判断（如数值是否 ≤ 0 、随机事件分发），表达力更强；

do-while 适合"先展示后验证"的交互场景，保证用户至少看到一次菜单；

for 适合"明确次数"的重复操作（如进度条 10 格、倒计时 3 秒）；

while 适合"条件驱动"的循环（如 `gameOver==0` 时继续每日循环）。

(2) break 与 continue 的语义差异：

break 表示"终止当前循环"——用于 switch case 退出和胜利后跳出日循环；

continue 表示"跳过本次迭代剩余部分"——用于死亡后跳过夜间阶段。

两者的语义差异在游戏逻辑中体现得非常直观。

(3) 结构清晰性：

在没有函数/数组/类的情况下组织约 500 行代码，变量命名和逻辑分块至关重要。

通过 ===== 分隔线注释 + 缩进层级控制 +

每段逻辑前的中文说明，保持了代码在单函数（main）内的可读性。

(4) 输入容错的价值：

`cin.fail() + cin.clear() + 消费坏输入的 while`

循环是实际编程中非常实用的模式，课本上很少讲到，但实际编程中必不可少。

不足与改进方向：

- 倒计时的延时使用空 for 循环，依赖于 CPU 速度，不够精确。后续学习后可改用 `sleep()`。
- 随机事件使用简单的 `rand()%N`，存在微小偏差。可升级为 C++11 `<random>` 库。
- 游戏平衡性（衰减速率、随机产出范围、夜间事件概率）可进一步调优。
- 后续学习了函数后可重构代码，将日循环、夜间事件等模块化。

总体而言，本次实验让我在实践中掌握了控制语句的选择与组合运用，为后续学习函数、数组、类和更复杂的程序结构打下了扎实的基础。